

TECHNISCHE UNIVERSITÄT
CHEMNITZ

Department of Electrical Engineering and
Information Technology

Chair of Measurement and Sensor Technology

Project Report

Project Lab Embedded Systems

Group:	11	
Members:	Komma Siva Vyshnavi Reddy	853746
	Sheikh Zeeshan Ahmed	852762
	Rushikesh Lakadkar	836920
	Randhir Bhagat	834322

Project:	Plant Vital Monitoring with Industrial Sensors
Date:	2025-07-09

Table of Contents

1	Abstract	1
2	Member Responsibilities.....	2
3	Functional Description	3
3.1	Overview.....	3
3.2	Hardware.....	4
3.2.2	Design Process.....	4
3.3	Software.....	14
3.3.1	LDC 1101 Teensy 4.0 Programming (Via SPI)	14
3.3.2	FDC2214 Teensy 4.0 Programming (Via I2C).....	15
4	Project Setup and Wiring Diagram.....	16
4.1	Interaction with users as well as hard- and software	18
5	Result of LDC1101 module	19
6	Current Status	21
6.1	Challenges	21
7	Conclusion	23
8	References.....	24
9	Annex.....	25
9.1	Project Code (LDC1101)	25
9.2	Code for FDC2214.....	29

Table of Images

Figure 1: LDC1101 Functional Block Diagram [1]	5
Figure 2: LDC1101 Schematic [3].....	6
Figure 3: LDC1101 PCB design with Eagle. [3].....	6
Figure 4: Assembled LDC1101 and Components. [3]	7
Figure 5: FDC2214 Functional Block Diagram [2]	8
Figure 6: FDC2214 Schematic [4]	9
Figure 7: FDC2214 PCB design with Eagle [4].....	9
Figure 8: Assembled FDC2214 and Components [4].....	10
Figure 9: Teensy 4.0 [6]	11
Figure 10: HW 689 Real Time Clock [6]	12
Figure 11: SD Card Adapter [6].....	13
Figure 12: BT05 Bluetooth Module [6].....	13
Figure 13: SPI Transaction Format [1]	14
Figure 14: Extended SPI Transaction [1]	14
Figure 15: I2C Write Register Sequence [2]	15
Figure 16: I2C Read Register Sequence [2]	15
Figure 17: Wiring connection with Hardware [5].....	16
Figure 18: Project setup with LDC1101 chip [5]	18
Figure 19: Data from Laptop [5]	19
Figure 20: Data from Mobile Application [5]	20
Figure 21: Used Logic Analyzer with FDC2214 [5].....	21
Figure 22: ACK bit was Missing in observation [5]	21
Figure 23: Fault in PCB Design [5]	22

Table of Tables

Table 1: Project Road Map	3
Table 2: Device Information LDC1101.....	5
Table 3: Device Information FDC2214.....	9
Table 4: Wiring Diagram Connections.....	17

1 Abstract

The project develops a system that uses industrial-grade sensors for monitoring plant vitals to advance precision agriculture and smart farming practices. The system tracks soil moisture levels along with temperature and humidity, and light intensity to evaluate plant health and growth conditions. The system processes collected data to identify early signs of stress or dehydration alongside environmental factors that are not optimal, thus enabling prompt intervention. Real-time data monitoring combined with reliable sensor technology enables this system to enhance agricultural decision processes. Greenhouses and vertical farms, as well as open-field agriculture, benefit from this system because it boosts crop production alongside resource optimization and sustainable agricultural results.

The project establishes an advanced plant monitoring system through industrial-grade capacitive and inductive sensors, which perform non-invasive, high-resolution environmental sensing. The system uses LDC1101 and FDC2214 converters to measure plant movement and moisture levels with exceptional precision and stability.

A specially designed PCB contains the sensor circuit while establishing efficient SPI and I²C connections to the Teensy 4.0 microcontroller. The system uses a Real-Time Clock (RTC) that allows precise time-stamping of all measurements for improved data acquisition. The system stores data on an SD card and enables wireless transmission through a Bluetooth module to support both offline data recording and immediate monitoring functions.

The system features easy deployment capabilities for various agricultural environments, which include greenhouses and controlled-environment laboratories, and small-scale farms. The platform provides continuous and precise plant condition data, which allows timely stress detection and enables data-driven agricultural decision making.

2 Member Responsibilities

Remark: The person responsible for each part needs to ensure that this part is done correctly until the deadline. He is NOT the only one working on that part.

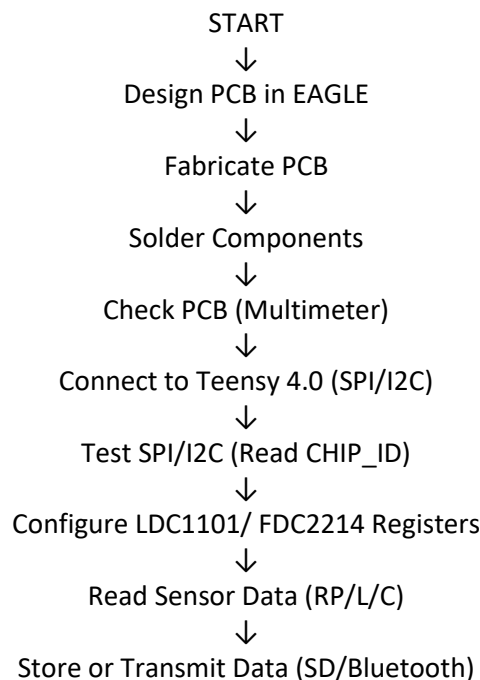
Randhir Bhagat:	Hardware Design (FDC2214) & Presentation
Rushikesh Lakadkar:	Hardware Design (LDC1101) & Documentation
Komma Siva Vyshnavi Reddy:	Configuration of Bluetooth, RTC, and SD Card
Sheikh Zeeshan Ahmed:	Configuration of FDC & LDC with Teensy 4.0 Setup Integration

3 Functional Description

3.1 Overview

This project is going to implement a smart, non-invasive system to monitor the health and condition of plants using sophisticated industrial sensors. The main sensors in this project are the inductance measurement chip LDC1101 and the capacitance measurement chip FDC2214. Both of these sensors collect a very broad range of data based on slight changes in the plant sediments' environmental factors. To be precise, the LDC1101 can detect plant stem motions and growth in the direction of the sensor. The FDC2214 sensor can detect moisture in the soil or moisture on the surface of the leaves, or in the air, without direct contact with the plant surface. The sensor reading is time-stamped via internal real-time clock (RTC) circuitry. All of the sensor data is recorded on an SD card for later review. It can also be accessed in real time by the USB or Bluetooth interface for monitoring or data collection on the computer or mobile device. The sensor system has been designed to be easy to set up, portable, and low-maintenance to be used in various environmental settings such as greenhouses, research laboratories, or small farms. It was also designed to be tailored and integrated quickly with other farming or monitoring systems.

Table 1: Project Road Map



3.2 Hardware

The hardware component of the project integrates electronic parts into a reliable, compact plant monitor. It focuses on non-invasive sensing, precise data logging, and efficient communication - all mounted onto a custom PCB for better durability and performance.

3.2.1. Requirements from the Application

The system needed to meet the following application-specific hardware requirements:

- High sensitivity: The system must detect subtle changes in capacitance and inductance associated with plant growth or environmental changes.
- Time-based data logging: requires a real-time clock to time-stamp each reading.
- Reliability of data storage and transmission: must allow for SD card logging, as well as transmission, wirelessly (Bluetooth) or via USB.

EAGLE was used for PCB design to provide a cleaner layout with less interference and greater stability than using a breadboard or loose jumper wires.

3.2.2 Design Process

The process of designing the hardware required choosing compatible components, testing each one operationally, and then arranging the components on a custom-designed PCB, to assemble a compact and reliable plant monitoring system. The PCB was laid out using Autodesk EAGLE, as EAGLE offered sufficient control over the layout and could ensure signal integrity defined by the project's requirements.

Design goals were maximizing the real estate on the PCB, minimizing noise, and allowing an easy connection to the plant environment.

1) LDC1101 (Inductance Sensor):

The LDC1101 is a high-resolution sensor that measures inductance and impedance in an LC circuit. It can detect tiny changes caused by the movement or presence of metal objects, making it great for non-contact sensing. It keeps the LC circuit oscillating and monitors how much energy is needed, helping it calculate the resonant frequency and parallel resistance (RP). The device also contains a comparator built into the device, which enables a digital signal to be issued if certain thresholds are exceeded. The sensor can be used with many parts, works for frequencies between 500 kHz to 10 MHz, and utilizes SPI for communications. requester approval. The device can operate from 1.8V to 3.3V and can incorporate the use of a single external capacitor for operation.

The resonant frequency of the sensor is

$$F_{sensor} = \frac{1}{2\pi\sqrt{L} * C}$$

Whereas,

L is the sensor inductance in Henrys,

C is the sensor parallel capacitance in Farads.

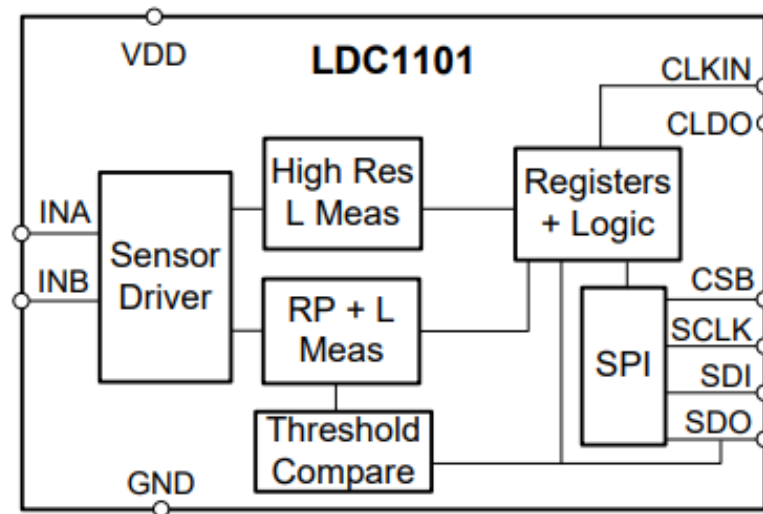


Figure 1: LDC1101 Functional Block Diagram [1]

Features of LDC1101

- Wide Operating Voltage Range: 1.8 V to 3.3 V
- Sensor Frequency Range: 500 kHz to 10 MHz
- Rp Resolution: 16-Bit
- L -Resolution:16-or24-Bit
- Operating Temperature: -40°C to $+125^{\circ}\text{C}$

Description

The LDC1101 is a small, high-accuracy sensor that transforms inductance into digital data. It operates at low voltages (1.8 V - 3.3 V) and is perfect for contactless sensing applications such as position, rotation, or motion detection in adverse conditions with dust or debris. Its significant ability is its dual-core measurement system, which allows the measuring of resistance (Rp) and inductance (L) to take place at an incredible rate greater than 150,000 samples per second for Rp and greater than 180,000 samples per second for L, with resolutions up to 24 bits. The sensor has also included a functionality for a real-time threshold comparison, which can be changed while the device is running, letting us build reactive real-time systems.

The chip can be utilized in several industries, including consumer electronics, industrial automation, automotive, and medical devices. It can record data such as movement, vibration, metal type, displacement, or pressure. When compared to traditional sensing, inductive sensing frequently offers improved resilience, cost-effectiveness, and immunity to environmental disturbances. All this power is contained in a tiny, 10-pin, 3 mm × 3 mm VSON package. The chip is easy to integrate into any embedded system because it uses a 4-pin SPI interface to communicate with microcontrollers.

Table 2: Device Information LDC1101

Part Number	Package	Body Size (NOM)
LDC 1101	VSON (10)	3.00 mm × 3.00 mm

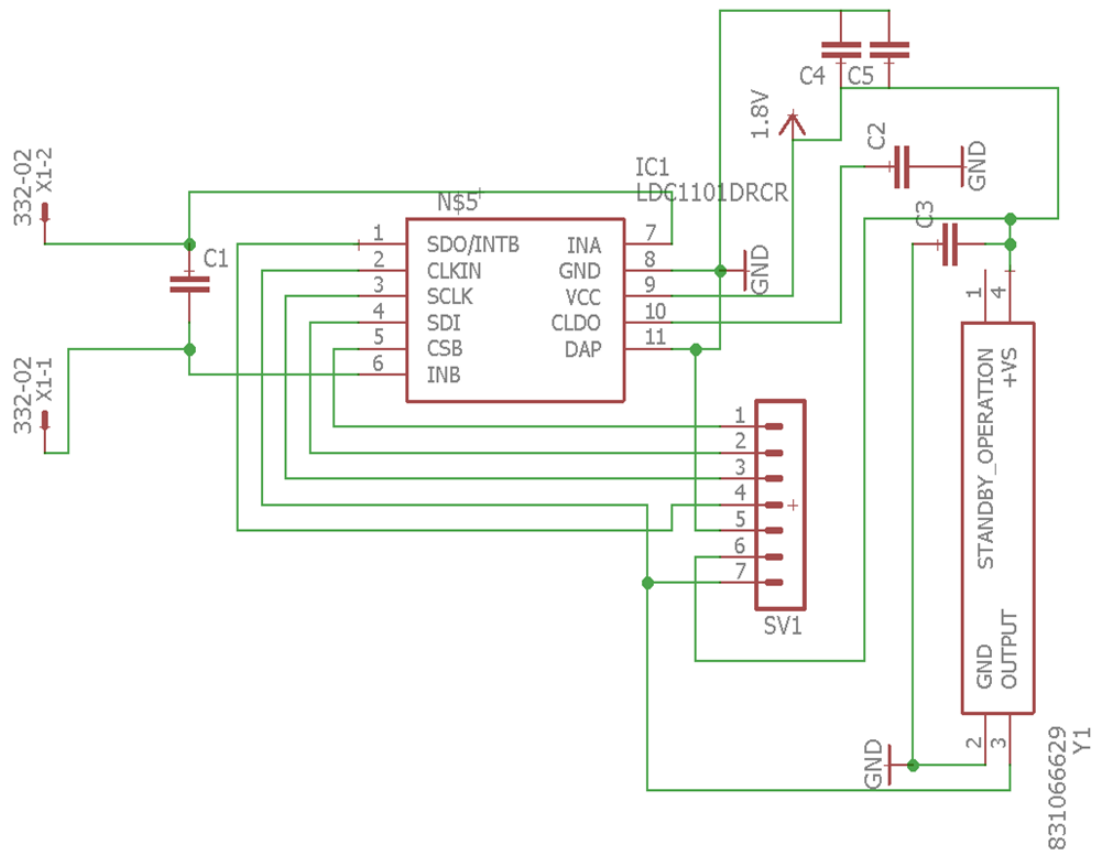


Figure 2: LDC1101 Schematic [3]

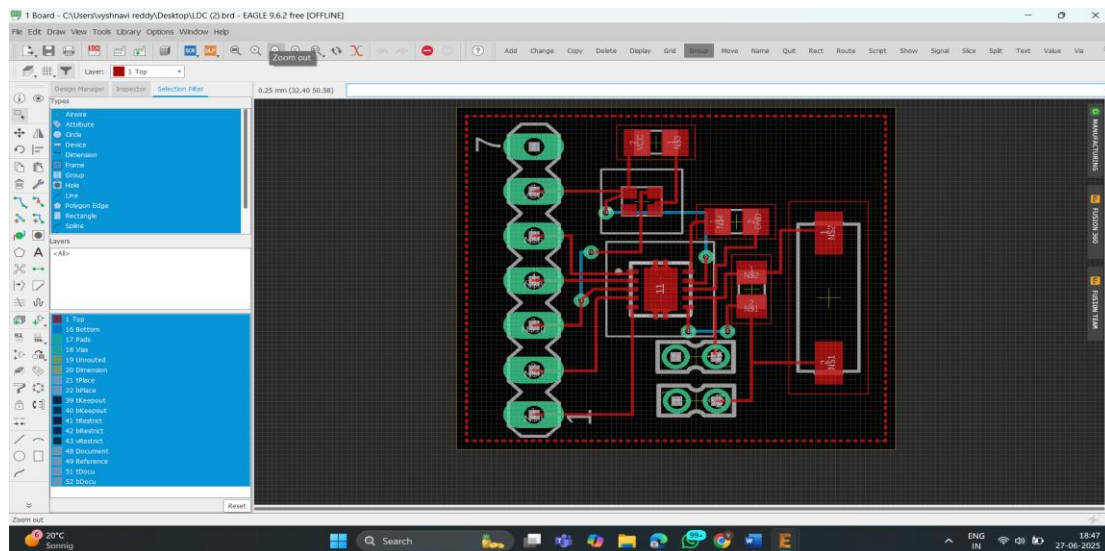


Figure 3: LDC1101 PCB design with Eagle. [3]

The LDC1101 sensor was placed centrally on the PCB to ensure stable signal routing and easy integration with the rest of the components.

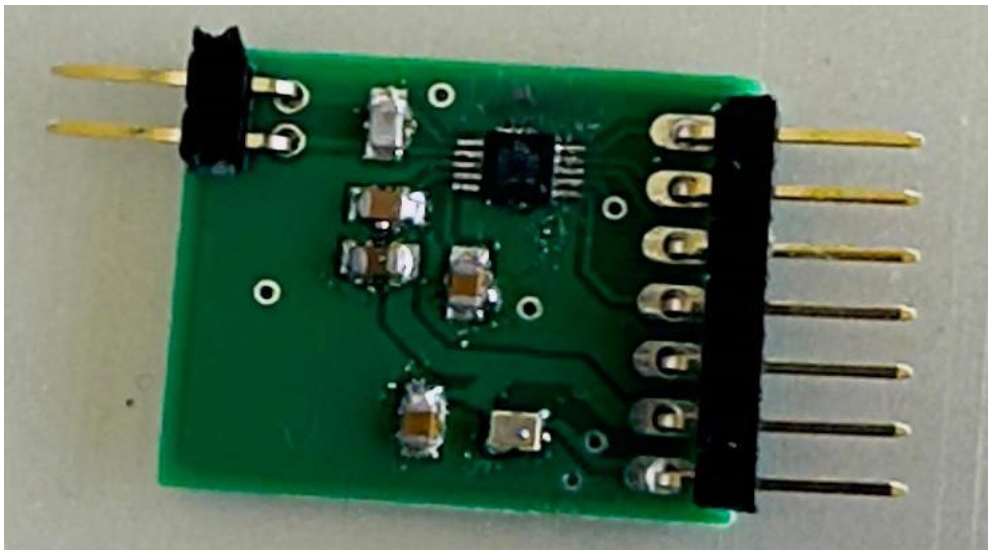


Figure 4: Assembled LDC1101 and Components. [3]

Applications

- High-Speed Gear Counting
- High-Speed Event Counting
- Motor Speed Sensing
- Knobs and Dials for Appliances, Automotive
- HMI for Appliances, Automotive, and Consumer Applications
- Motor Control
- Metal Detection

2) FDC2214 (Capacitance Sensor):

The FDC2214 is a very precise sensor chip that converts capacitance into digital data. It's an excellent tool for creating a capacitive sensing system, such as touch sensing, water sensing, and even moisture detection, because it allows for a high level of accuracy. The FDC2214 differs from capacitive sensors in the past that switch charge around; it uses an LC resonator (inductor and capacitor paired to sense). The LC circuit of an inductor and capacitor is sensitive to changes, for example, when the capacitance changes (or in some cases, a leaf moves closer) or moisture changes -- the resonant frequency of the LC circuit changes. The chip then plots and detects this frequency shift and turns that into digital values.

The FDC2214 is much more capable in a noisy or cluttered environment than conventional capacitive sensors due to its narrow-band design and virtually insensitivity to electrical noise (EMI).

In summary, the FDC2214 provides a clean real capacitance measurement because of the way it detects capacitance as a function of frequency, and you can use this data to measure changing moisture levels, plant movement, and environmental conditions.

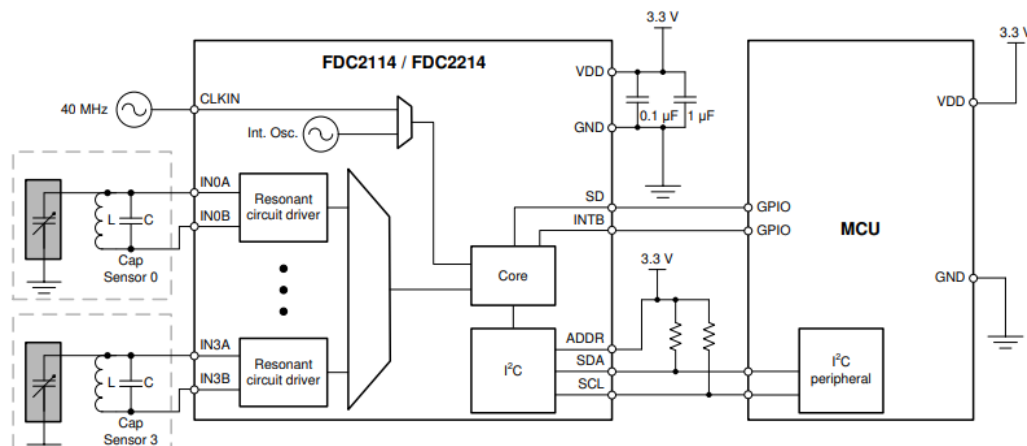


Figure 5: FDC2214 Functional Block Diagram [2]

Features

- Resolution: up to 28 bits
- Supply voltage: 2.7V to 3.6V
- Power consumption: active: 2.1mA
- Temperature range: -40°C to $+125^{\circ}\text{C}$

Description

The low-power, contactless technique known as capacitive sensing is capable of detecting liquid levels, gestures, and proximity. It gives a great deal of a flexible and low-power technology. Capacitive sensing makes it simple to detect liquid levels, gestures, and proximity. Designers may create systems with a variety of materials thanks to capacitive sensing, which uses any conductive object—like metal—as the sensing element. Electrical noise is one element of capacitive sensing systems that may cause problems and lower accuracy. Even in loud surroundings (such as under fluorescent lights), the resonant sensing architecture of the FDC2214 series (the FDC2214, for example) minimizes noise, making those difficulties tolerable.

These devices, which utilize chips such as the FDC2214 and FDC2114, are multi-channel, high-resolution, and high-speed sensors, allowing simultaneous measurements with numerous sensors at high rates of accuracy that are still fast enough to perform low-latency measurements. Importantly, the chips act over a broad frequency range, which is particularly useful for sensing conductive fluids like soap, ink, and detergents. The FDC2214 models are rated for very high accuracy (up to 28-bit accuracy) while the FDC2114 models are designed for rapidly sensing objects (up to 13.3k samples per second), which is useful when resourcing fast-moving objects.

They are ideal for remote sensors and for tracking gradual changes in the environment, such as temperature or humidity, because they can also operate with large input capacitances, up to 250nF. All things considered, the FDC2214 family is appropriate for liquid level monitoring and proximity sensing, especially for conductive liquids. Special shielding makes a chip like the FDC1004 more suitable in non-conductive liquids and situations with many potential points of interference (like human hands).

Table 3: Device Information FDC2214

Part Number	Package	Body Size (NOM)
FDC 2214	RGH (WQFN, 16)	4.00 mm × 4.00 mm

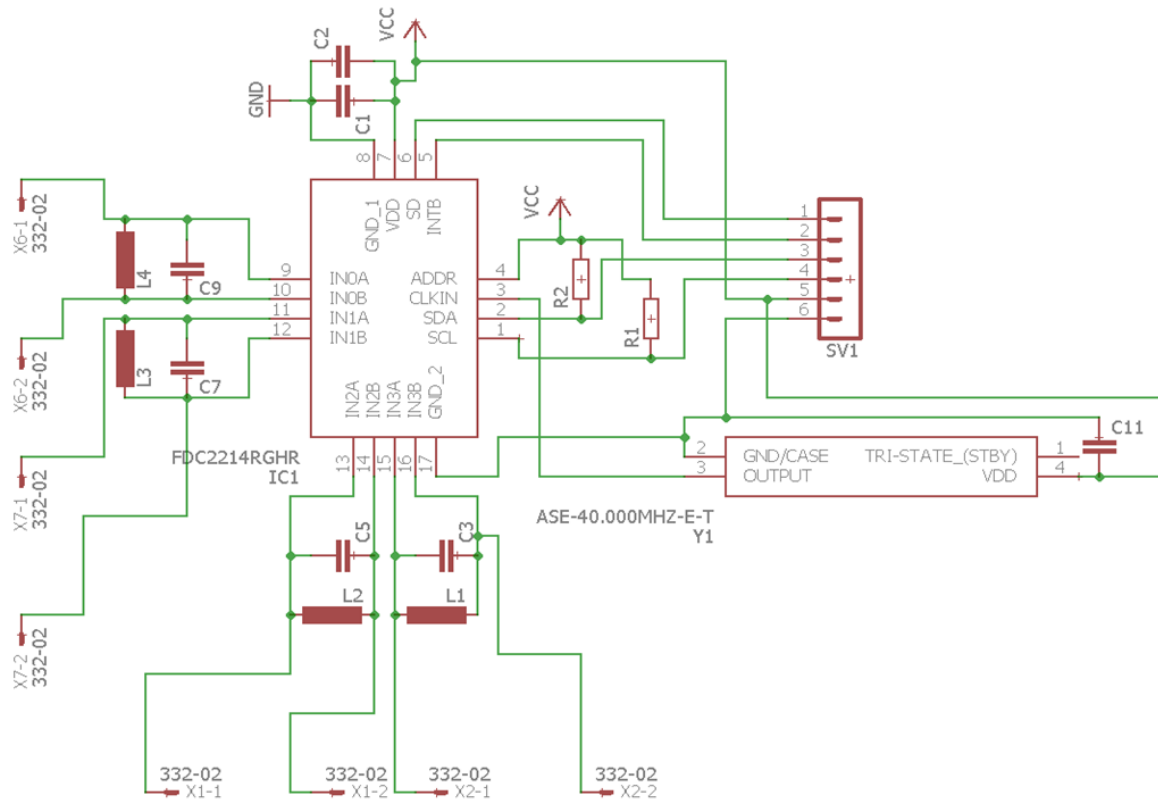


Figure 6: FDC2214 Schematic [4]

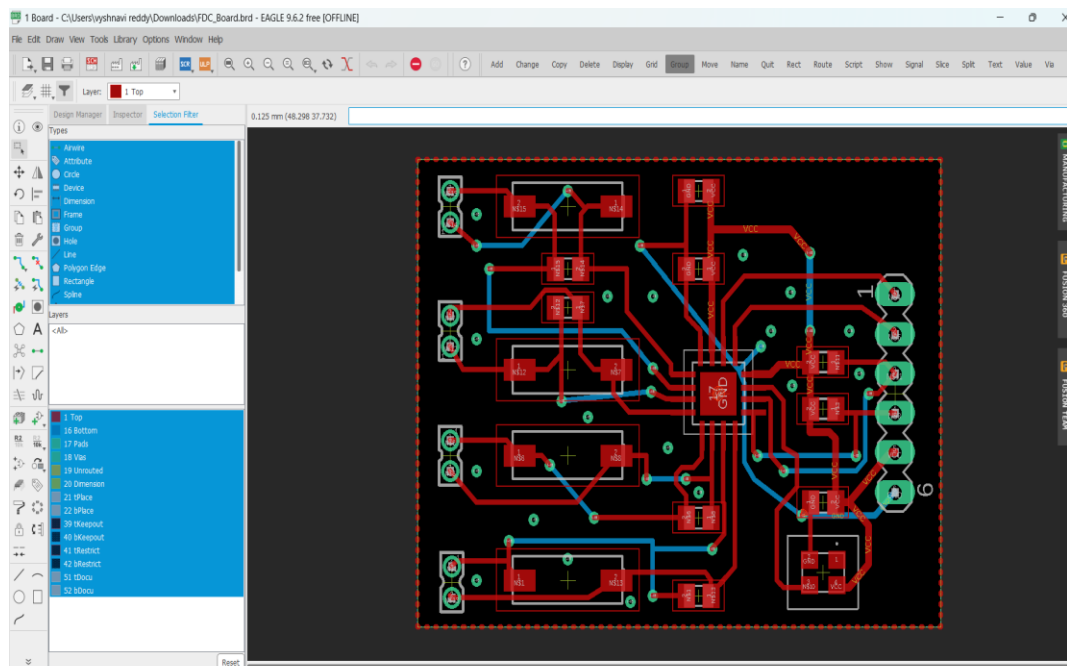


Figure 7: FDC2214 PCB design with Eagle [4]

Mounted near the centre of the PCB, this sensor connects to external electrodes placed in soil or on leaves. Central placement allows flexible wiring to different electrode setups and reduces noise interference.

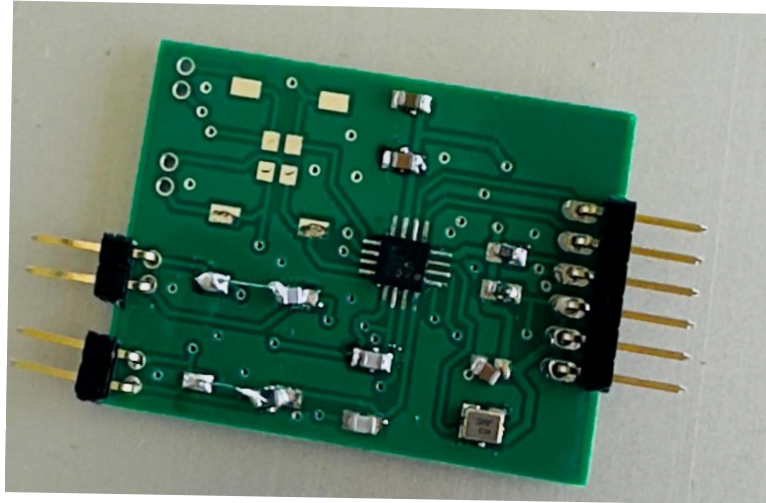


Figure 8: Assembled FDC2214 and Components [4]

Applications

- Proximity sensor
- Gesture recognition
- Level sensor for liquids, including conductive ones such as detergent, soap, and ink.

3) Teensy 4.0 (Microcontroller):

The Teensy 4.0 acts as the central processing unit of the entire system. It is responsible for reading data from both the LDC1101 (inductance sensor) and the FDC2214 (capacitance sensor) via SPI and I²C communication protocols, respectively.

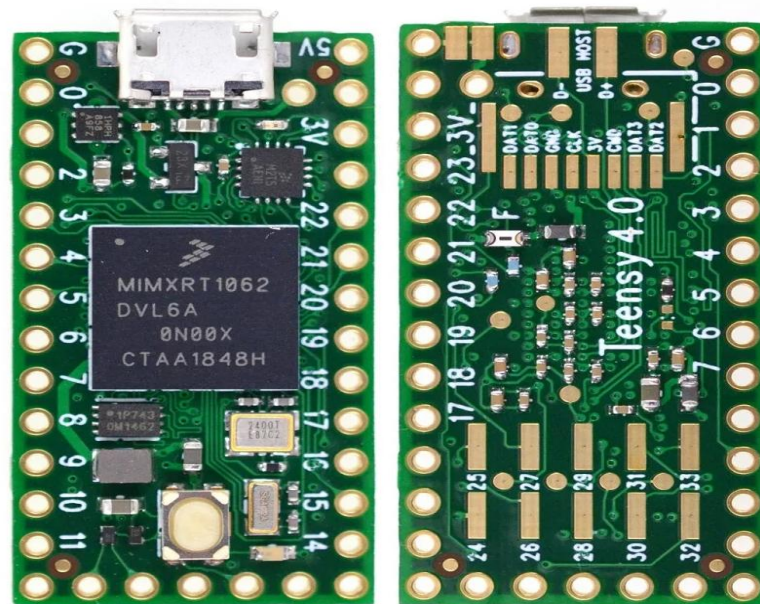


Figure 9: Teensy 4.0 [6]

At the heart of its operation is the collection of sensor data. It interfaces with two key industrial sensors:

- The LDC1101, an inductance-to-digital converter, sends data to the Teensy using SPI (Serial Peripheral Interface), allowing the Teensy to read a precise inductance value that relates to changes in the plant, such as stem expansion or movement.
- The FDC2214, a capacitance-to-digital converter, communicates using I²C (Inter-Integrated Circuit), allowing the Teensy to read capacitance values that could indicate changes in moisture, the proximity of leaves, or the humidity of the environment.

In addition to reading sensor data, the Teensy also:

- Timestamps each data reading using information from the RTC module (HW-689).
- Records the data in real-time to the SD card module and stores it in an organized CSV format for analysis.
- Transmits data wirelessly through the HC-05 Bluetooth module, so the system can stream data to a mobile device/computer.

4) HW689 (RTC Module):

The HW689 real-time clock is located near the microcontroller. This module provides accurate timestamps, which are extremely valuable for data logging and trend analysis.

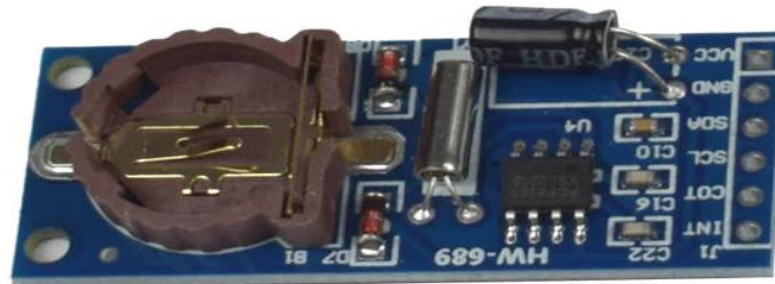


Figure 10: HW 689 Real Time Clock [6]

Placed near the Teensy 4.0 microcontroller on the PCB, this Real-Time Clock (RTC) guarantees that every piece of sensor data — whether from the LDC1101 inductance sensor or the FDC2214 capacitance sensor — is logged with an exact time and date of collection. This functionality is important for long-term plant monitoring when changes in a plant's condition over periods of hours, days, and weeks are needed for good analysis.

HW689 is a battery-backed unit, which means it continues to keep accurate time even if the primary system is powered down. This makes it extremely reliable for practical uses in the real world, including greenhouse environments or field deployments where the power may not be stable.

The Teensy communicates in periodic intervals with the HW689 via the I²C protocol to pull timestamps; the timestamps (when) are then stored with the sensor data on the microSD card used for historical logging. Broadcast in real-time via Bluetooth or USB to external devices for live monitoring. Evaluated to explore trends like daily plant activity, decay of moisture over time, or patterns of response to environmental scenarios.

5) Micro SD Card Adapter:

Positioned on the PCB edge for easy access, the micro-SD card adapter is responsible for storing collected sensor data.

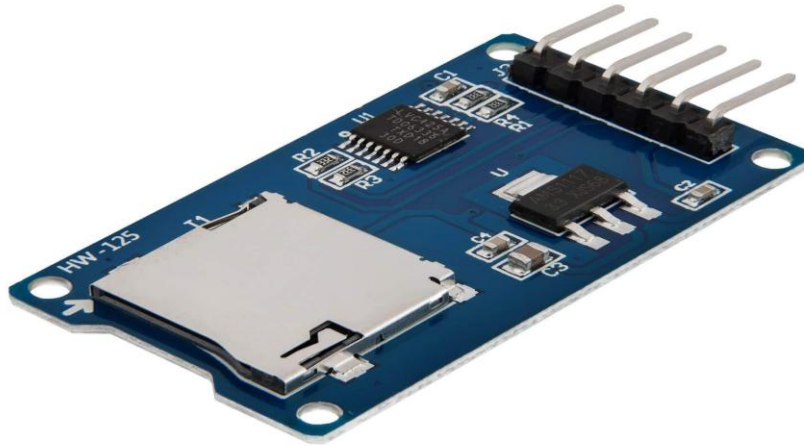


Figure 11: SD Card Adapter [6]

This module receives data directly from the Teensy 4.0 microcontroller, which regularly collects readings from the LDC1101 inductance sensor and the FDC2214 capacitance sensor, along with timestamps from the HW689 RTC module. The data is written onto the SD card in CSV (Comma-Separated Values) format.

6) BT05 Bluetooth Module:

Mounted near the communication ports of the PCB, the BT05 module facilitates wireless data transmission to smartphones or PCs.

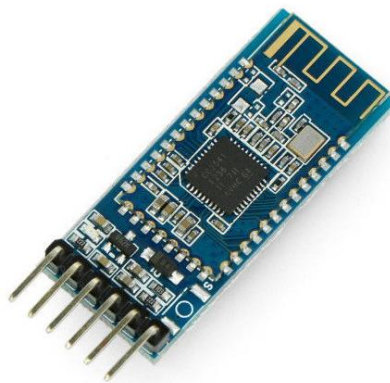


Figure 12: BT05 Bluetooth Module [6]

3.3 Software

The Teensy 4.0 Integrated Development Environment (IDE) is the main software used to program Teensy 4.0 microcontrollers. It lets users write code in a simplified C/C++ language, compile it, and upload it directly to the Teensy 4.0 board. The IDE has a text editor, message area, serial monitor, and built-in libraries for connecting with sensors, modules, and communication protocols like SPI and I²C. In our project, we used the Teensy 4.0 IDE to connect with sensors (like LDC1101 and FDC2214), process the data, and control hardware components.

3.3.1 LDC 1101 Teensy 4.0 Programming (Via SPI)

The LDC1101 uses SPI to set up the internal registers. You need to configure the LDC1101 while it is in Sleep mode. If you need to change a setting on the LDC1101, put the device back in Sleep mode, update the right register, and then switch the LDC1101 back to conversion mode. CSB must go low before you can access the first address. If the number of SCLK pulses is less than 16, a register write command will not change the contents of the addressed register.

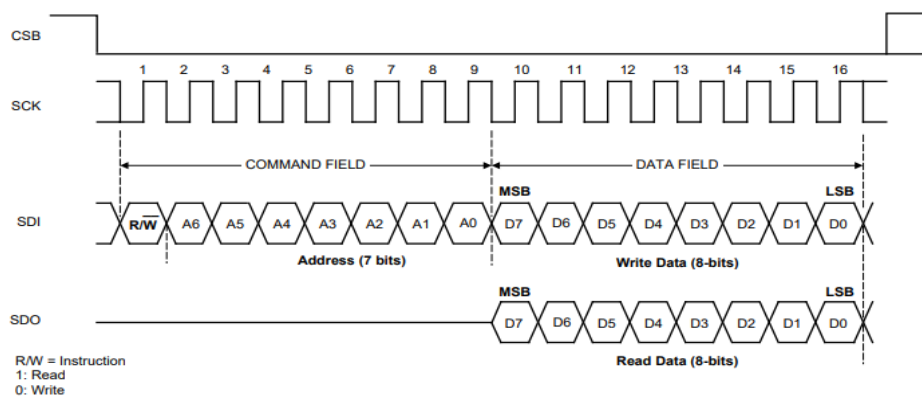


Figure 13: SPI Transaction Format [1]

The LDC1101 supports a longer SPI transaction where CSB stays low, and you can write or read sequential register addresses. After the first register transaction, each extra 8 SCLK pulses targets the next register. It reads or writes depending on the initial R/W flag in the original command. A register writes command takes effect on the 8th clock pulse. You can program two or more registers this way. The register address must not go above 0x3F.

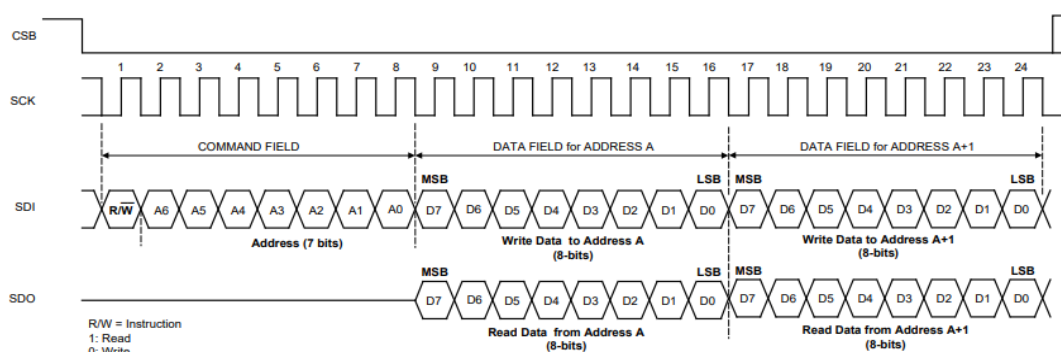


Figure 14: Extended SPI Transaction [1]

- **Programming for LDC1101**

This Teensy 4.0 program reads high-resolution inductive sensor data from the LDC1101 using SPI communication, and timestamps it using a real-time clock via I²C. It uses libraries like SPI.h, Wire.h, SdFat.h, and RTCLib.h to manage sensor communication, data logging, and timekeeping.

3.3.2 FDC2214 Teensy 4.0 Programming (Via I2C)

The FDC communicates register access through an I2C protocol, which starts by extending its start sequence. The I2C interface achieves its highest operational speed at 400kbps. The sequence starts with the standard I2C 7-bit target address, followed by an 8-bit pointer register byte that determines the register address. The FDC I2C address becomes 0x2A when the ADDR pin is low while the address changes to 0x2B when the ADDR pin is high. The ADDR pin must remain stable after the FDC leaves Shutdown Mode because changing its state causes problems.

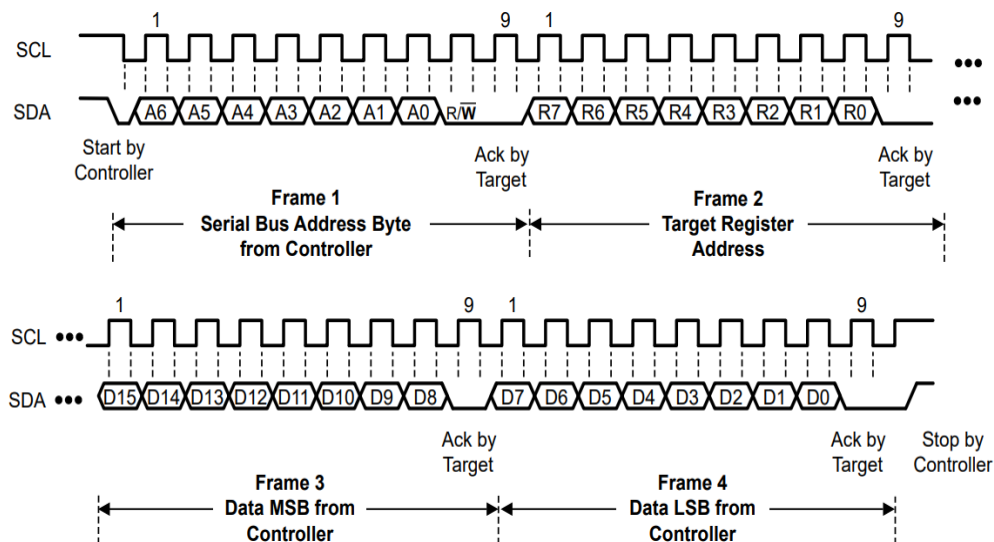


Figure 15: I2C Write Register Sequence [2]

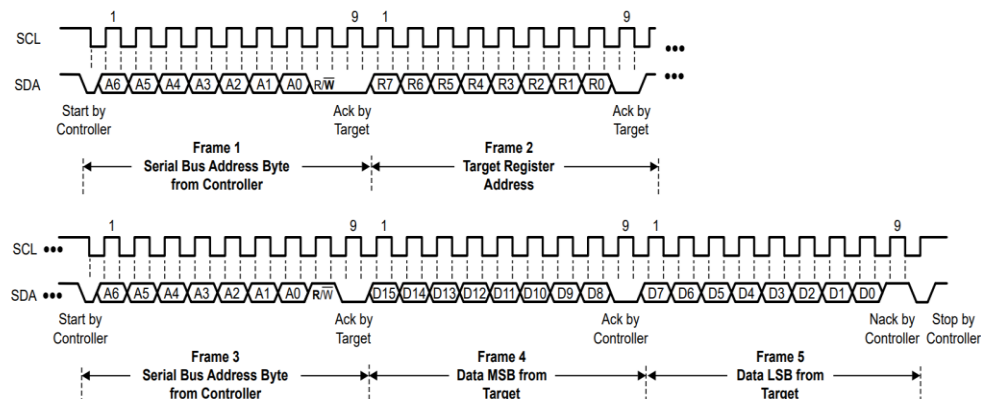


Figure 16: I2C Read Register Sequence [2]

- **Programming for FDC2214**

This Teensy 4.0 code connects with the FDC2214 capacitive sensor using the I²C protocol. It sets up Channel 3 of the sensor for continuous data collection, including parameters like sensor drive current, RCOUNT, and clock dividers. The code reads raw capacitance values from Channel 3 and displays the sensor identification registers (MANUFACTURER_ID and DEVICE_ID) on the Serial Monitor for verification. This arrangement enables high-resolution capacitive sensing, making it useful for proximity, liquid level, or touch detection applications

4 Project Setup and Wiring Diagram

Built a Smart Plant Monitoring Prototype using industrial-grade sensors (LDC1101 and FDC2214) on a breadboard with Teensy 4.0, Bluetooth, and SD card storage. This setup allows for real-time environmental data collection and wireless communication.

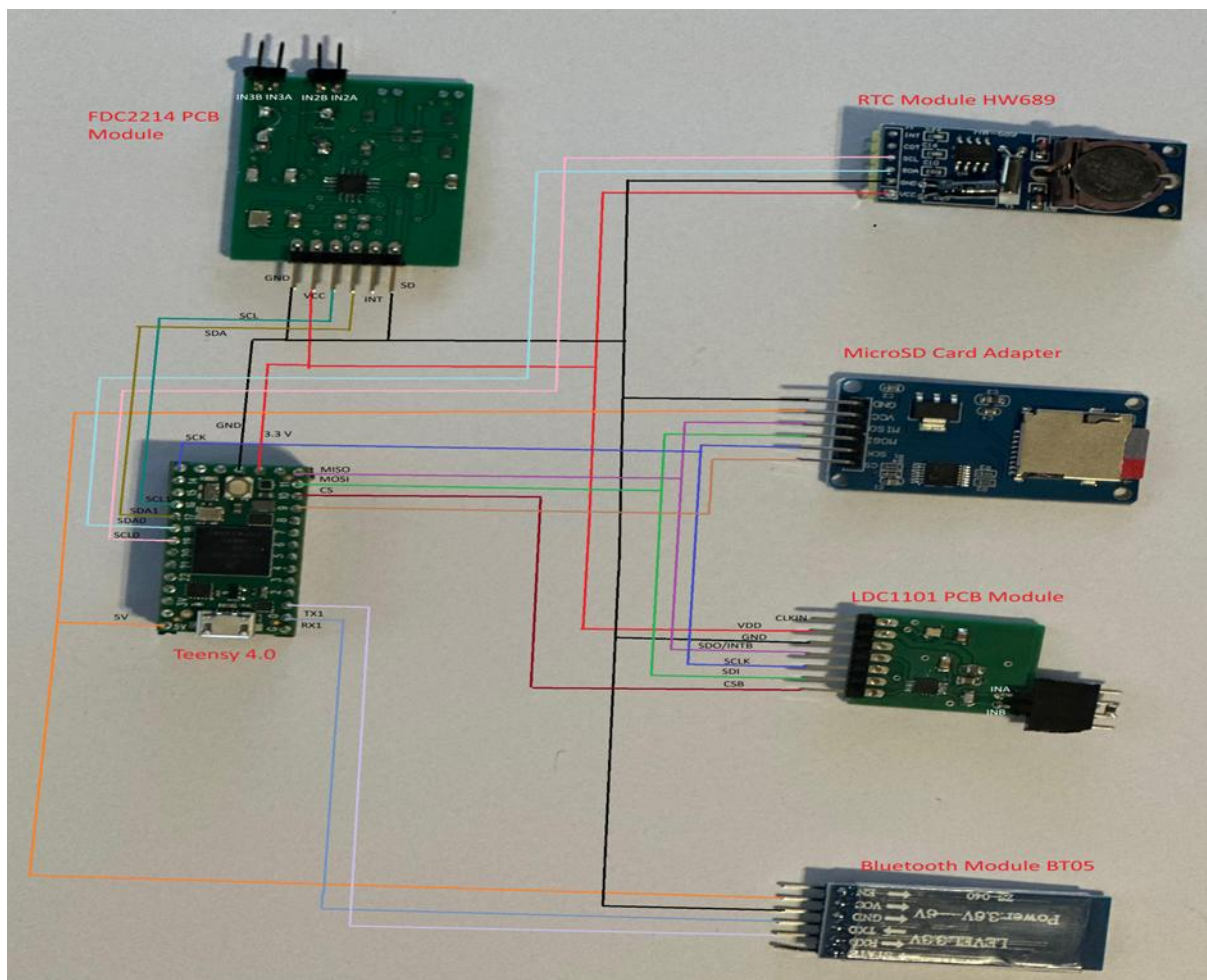


Figure 17: Wiring connection with Hardware [5]

Table 4: Wiring Diagram Connections

Module Names	Module Pins	Teensy 4.0
RTC Module HW689	SCL	SCL0 (Pin 19)
	SDA	SDA0 (Pin 18)
	GND	GND
	VCC	3.3V
MicroSD Card Adapter	GND	GND
	VCC	5V
	MISO	MISO (Pin 12)
	MOSI	MOSI (Pin 11)
	SCK	SCK (Pin 13)
	CS	Pin 9
LDC1101 PCB Module	VDD	3.3V
	GND	GND
	SDO/INTB	MISO (Pin 12)
	SCLK	SCK (Pin 13)
	SDI	MOSI (Pin 11)
	CSB	CS (Pin 10)
Bluetooth Module BT05	VCC	5V
	GND	GND
	TXD	RX1 (Pin 0)
	RXD	TX1 (Pin 1)
FDC2214 PCB Module	GND	GND
	VCC	3.3V
	SCL	SCL1 (Pin 16)
	SDA	SDA1 (Pin 17)
	SD	GND

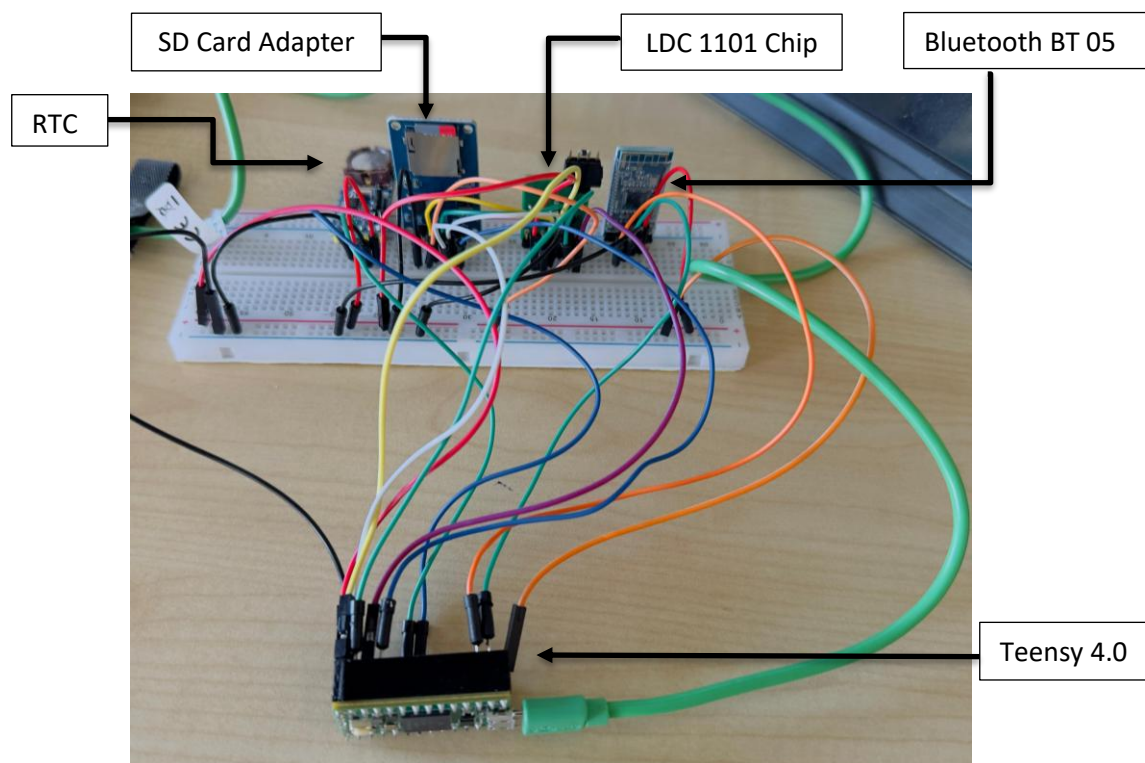


Figure 18: Project setup with LDC1101 chip [5]

4.1 Interaction with users as well as hard- and software

Real-time sensor data was received on a mobile application through the BT05 Bluetooth module. This enabled smooth wireless communication between the monitoring system and a smartphone. Users could connect their devices using available apps to see live readings of key environmental factors, like soil moisture, temperature, humidity, and plant inductance variations. This allowed them to monitor the health and condition of plants from a distance without needing to interact physically with the hardware. Users could also record, study, and understand trends in plant behavior and environmental changes. The mobile app made the system very user-friendly and suitable for real-world use in smart farming and research.

5 Result of LDC1101 module

The system successfully captured and transmitted real-time data from the LDC1101 inductance sensor module to both mobile devices and computers.

The resonant frequency of the sensor is

$$F_{sensor} = \frac{1}{2\pi\sqrt{L}*C} \quad (1)$$

We have, CLKIN = 16 MHz and C = 820 nF

LHR output (Measured) = 1.801089 MH

$$\begin{aligned} F_{sensor} &= \frac{LHRoutput}{2^{24}} \times CLKin \\ &= \frac{1801089}{2^{24}} \times 16 \times 10^6 \\ F_{sensor} &= 1.717 \text{ MHz} \end{aligned}$$

Put F_{sensor} Value in Equation (1)

$$\begin{aligned} F_{sensor} &= \frac{1}{2\pi\sqrt{L} * C} \\ L &= 10.5 \text{ nH} \end{aligned}$$

```
*****SD Card Data*****
LHRData: 1897600    Time: 15:0:0    Date: 30/6/2025
LHRData: 1630428    Time: 15:0:1    Date: 30/6/2025
LHRData: 1892504    Time: 15:0:2    Date: 30/6/2025
LHRData: 1893464    Time: 15:0:3    Date: 30/6/2025
LHRData: 1892584    Time: 15:0:5    Date: 30/6/2025
LHRData: 1884330    Time: 15:0:6    Date: 30/6/2025
LHRData: 1897529    Time: 15:0:7    Date: 30/6/2025
LHRData: 1892553    Time: 15:0:8    Date: 30/6/2025
LHRData: 1892458    Time: 15:0:9    Date: 30/6/2025
*****SD Card END*****
```

Figure 19: Data from Laptop [5]

To observe real-time data without cables from our sensor system, we used the BlueLight app - a simple and easy Bluetooth terminal application for Android devices.

The app allows for a connection with Bluetooth modules like the HC-05 (which we used), and once paired, it can show live data that is sent from the microcontroller using UART. For instance, in our project, the app receives time-stamped sensor data from inductive and capacitive sensors. It allows us to see the status of plant health variables in real-time and transmit the data without needing a wired connection or a computer.

The benefits of the BlueLight app in our project have been:

- Real-time data viewing via Bluetooth.
- Quick and easy pairing with the HC-05 module.
- No additional setup, just connect and receive.
- Remote data monitoring can be done with any Android smartphone.

The BlueLight app made the entire system easier to use and more practical, especially when being used in the field and or a greenhouse where a PC isn't accessible.

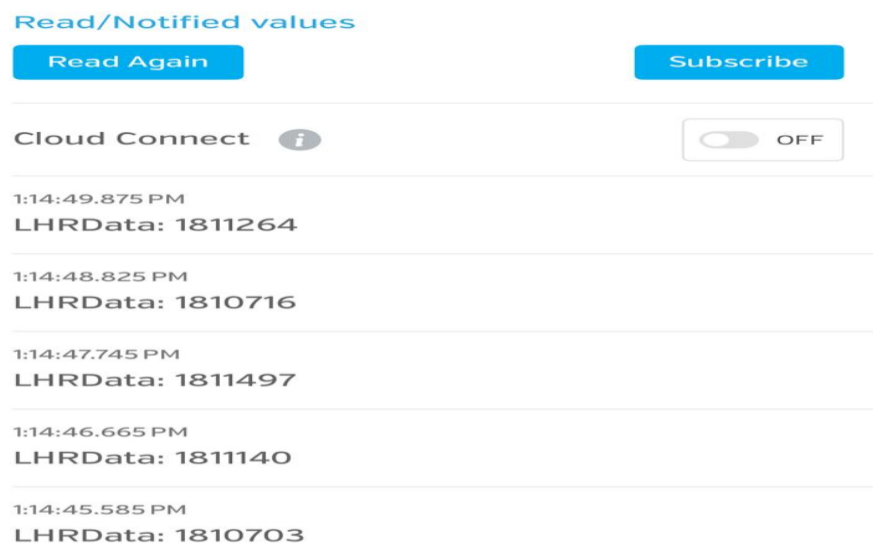


Figure 20: Data from Mobile Application [5]

6 Current Status

The current stage of our project shows that we have successfully integrated hardware and software. The LDC1101 inductive sensor is connected with the Teensy microcontroller through SPI communication, which allows us to gather LHR data accurately. The RTC module is also communicating properly with the system via I²C and providing real-time timestamps for sensor data. We have completed successful read and write operations on the SD card, which ensures reliable data logging. Additionally, the project enables wireless data transmission through Bluetooth, allowing us to monitor sensor readings in real time.

6.1 Challenges

Initially, we used a logic analyzer to diagnose the communication issue with the FDC2214 sensor. During analysis, we observed that the acknowledgment (ACK) bit was missing in the I²C response, indicating that the sensor was not responding to the address sent by the microcontroller.

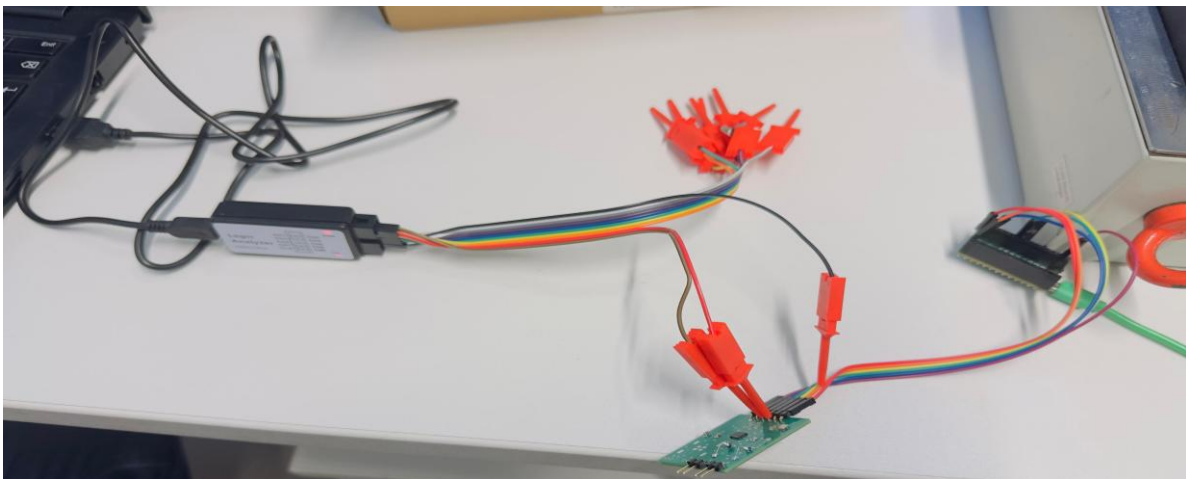


Figure 21: Used Logic Analyzer with FDC2214 [5]

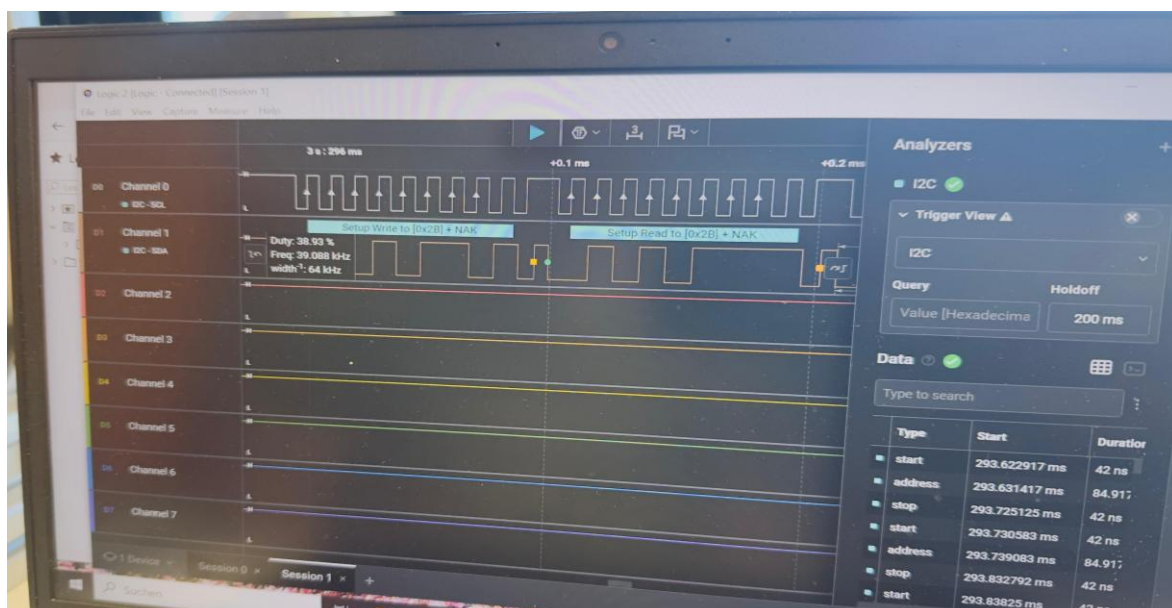


Figure 22: ACK bit was Missing in observation [5]

After looking more closely, we found a fault in the PCB design that led to wrong connections at the I²C lines. Once we fixed the design, we created an updated PCB layout that solved the communication issue.

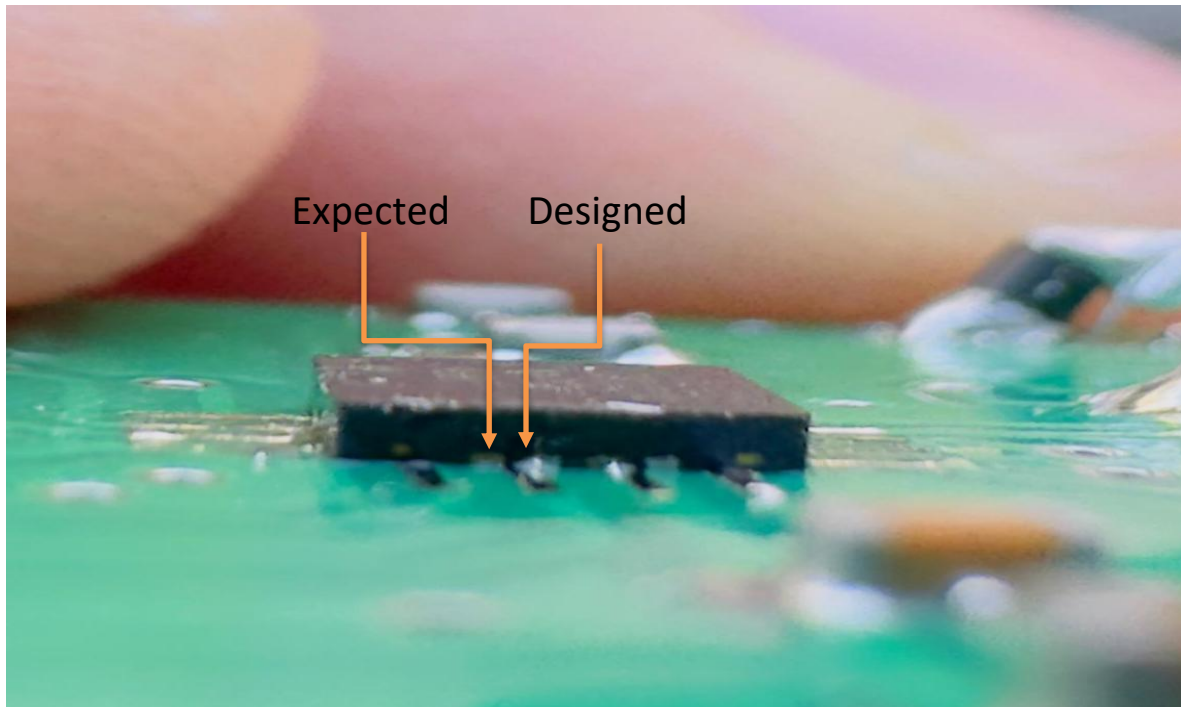


Figure 23: Fault in PCB Design [5]

7 Conclusion

In this project, we were successful in designing an intelligent plant monitoring system using industrial-grade capacitive and inductive sensing technologies. By combining the features of LDC1101 and FDC2214 sensors on a specially designed PCB, we created a small, precise platform for non-invasive, non-contact plant health monitoring of parameters such as stem motion and humidity content.

The system utilizes the Teensy 4.0 microcontroller's high-speed processing powers and offers reliable time-logging through using an onboard Real-Time Clock (RTC) module. The system offers wireless (Bluetooth) as well as local (SD card) data storage, offering flexibility across a variety of deployment scenarios — from controlled settings like greenhouses and laboratory facilities to field applications.

While the sensors initially were not able to communicate with the PCB because of some design problems, timely troubleshooting and redesign enabled the successful integration of the sensors and valid data acquisition. Our final system offers timed, valid environmental data in real-time, which can be used for monitoring trends in plant activity and farm condition optimization.

This project demonstrates a robust foundation for smart agriculture applications, opening up the way for scalable systems that can lead to sustainable and data-driven farming solutions.

8 References

1. Texas Instruments Incorporated. (2016). LDC1101 1.8-V High-Resolution, High-Speed Inductance-to-Digital Converter [Technical manual].
<https://www.ti.com/lit/ds/symlink/ldc1101.pdf>
2. FDC2214 data sheet, product information and support | TI.com. (n.d.).
<https://www.ti.com/product/FDC2214>
3. LDC1101 - Google Drive. (n.d.). Google Drive.
https://drive.google.com/drive/folders/1IrN6HWdk9oRrfM9LKO4tfMBH7DWAbs8?usp=drive_link
4. FDC2214 - Google Drive. (n.d.). Google Drive.
https://drive.google.com/drive/folders/1Vq0Tc-MECMADvFhva9oh7vDhHj89oiy?usp=drive_link
5. Setup photos - Google Drive. (n.d.). Google Drive.
https://drive.google.com/drive/folders/1DPBWYxrA6F-4h2PfMvFqMYst3x2fYfnp?usp=drive_link
6. Components - Google Drive. (n.d.). Google Drive.
<https://drive.google.com/drive/folders/1XJp2vHJehz3JfGdg7PYrKDumUkCGjrt?usp=sharing>

9 Annex

9.1 Project Code (LDC1101)

```
#include <SPI.h>
#include <SdFat.h>
#include <Wire.h>
#include "RTCLib.h"

SdFat SD; //SD card variable
RTC_PCF8563 rtc; // Create RTC object
const int chipSelect_SDCard = 9; //CS for SD Card
const int CS_PIN_LDC = 10; // Chip Select pin for LDC1101
#define BTSerial Serial1 // Bluetooth via Serial1 (Pins 0 and 1)
#define READ_REG 0x80 //read enable byte for LDC
#define WRITE_REG 0x00 //write enable byte for LDC
//write operation for LDC
void writeRegister_LDC(uint8_t reg, uint8_t value) {
    digitalWrite(CS_PIN_LDC, LOW); //enable CS of LDC1101, select as slave
    SPI.transfer(WRITE_REG | (reg & 0x3F)); // write command, enable write operation, register
    address must not increment above 3F
    SPI.transfer(value); //write data
    digitalWrite(CS_PIN_LDC, HIGH); //disable CS of LDC1101, deselect slave
}
//read operation for LDC
uint8_t readRegister_LDC(uint8_t reg) {
    digitalWrite(CS_PIN_LDC, LOW); //enable CS of LDC1101, select as slave
    SPI.transfer(READ_REG | (reg & 0x3F)); // read command, enable read operation, register
    address must not increment above 3F
    uint8_t val = SPI.transfer(0x00); //read data
    digitalWrite(CS_PIN_LDC, HIGH); //disable CS of LDC1101, deselect slave
    return val;
}
void LHR_modeSetup()
{
    SPI.begin();
```

```

    SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0)); // 1 MHz, Mode 0 (set
Clock idle low, sample on rising edge), send MSB first
    pinMode(CS_PIN_LDC, OUTPUT); // set CS pin as output
    digitalWrite(CS_PIN_LDC, HIGH); // set LDC1101 slave, disable mode
    // Device in Power-On Reset – device should only be configured when POR_READ = 0.
    while(readRegister_LDC(0x20) & 0x01); // Device is currently in reset; wait until POR_READ =
0, Loop while bit 0 == 1
    writeRegister_LDC(0x01, 0x05); // RPMIN = 3 kΩ
    writeRegister_LDC(0x0B, 0x01); // (START_CONFIG) activate sleep mode, register contents are
maintained
    writeRegister_LDC(0x34, 0x00); // Sensor Frequency not divided (default value)
    // Set LHR_RCOUNT to 0xFFFF for high resolution
    writeRegister_LDC(0x30, 0xFF); // LHR_RCOUNT_LSB
    writeRegister_LDC(0x31, 0xFF); // LHR_RCOUNT_MSB
    // set offset to LHR data 0x0000
    writeRegister_LDC(0x32, 0x00); // LHR_OFFSET_LSB
    writeRegister_LDC(0x33, 0x00); // LHR_OFFSET_MSB
    // configuration LHR mode
    writeRegister_LDC(0x05, 0x01); // ALT_CONFIG.LOPTIMAL(register 0x05-bit0) = 1
    writeRegister_LDC(0x0C, 0x01); // D_CONFIG.DOK_REPORT (register 0x0C-bit0) = 1
    writeRegister_LDC(0x0B, 0x00); // (START_CONFIG) disable sleep mode
    Serial.print("LHR Setup Completed\n");
}

uint32_t read_LHR_Data()
{
    SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0)); // 1 MHz, Mode 0 (set
Clock idle low, sample on rising edge), send MSB first
    while (readRegister_LDC(0x3B) & 0x01); // Determine LHR data ready, (LHR Status Register)
    Loop while bit 0 == 1
    uint8_t lsb = readRegister_LDC(0x38); // LHR_DATA_LSB
    uint8_t mid = readRegister_LDC(0x39); // LHR_DATA_MID
    uint8_t msb = readRegister_LDC(0x3A); // LHR_DATA_MSB
    uint32_t lhrValue = ((uint32_t)msb << 16) | ((uint32_t)mid << 8) | lsb;
    return lhrValue;
}

```

```
void SDcard_setup()
{
    pinMode(chipSelect_SDcard, OUTPUT); // set CS for SDcard as output
    digitalWrite(chipSelect_SDcard, HIGH); //sd card CS as high
    if (!SD.begin(chipSelect_SDcard, SD_SCK_MHZ(10))) { // Force SPI speed to 10 MHz
        Serial.println("Card failed or not present.");
        while (1);
    }
    Serial.println("Card initialized.");
}

void setup() {
    Serial.begin(9600);
    while (!Serial);
    BTSerial.begin(9600); // BT-05 default baud rate
    Wire.begin(); // Initialize I2C, module as Master
    //check the connection of RTC
    if (!rtc.begin()) {
        Serial.println("Couldn't find RTC");
        while (1);
    }
    // Set time (only run once to set)
    rtc.adjust(DateTime(2025, 6, 29, 13, 25, 0)); //set starting date
    Serial.println("\nRTC is running.");
    SDcard_setup(); //call SD Card setup function
    LHR_modeSetup(); //Call LHR setup function
}

void loop() {
    Serial.print("\n"); //Display
    uint32_t lhrValue = read_LHR_Data();
    //Read Time Value
    DateTime now = rtc.now();
    BTSerial.print("LHRData: " + String(lhrValue)); //send data on bluetooth
    // ----- WRITE in FILE -----
```

```
FsFile file = SD.open("Data7.txt", FILE_WRITE);//open text file in SD card
if (file) {
    file.println("  LHRData: " + String(lhrValue) + "    Time: " + String(now.hour()) + ":" +
String(now.minute()) + ":" + String(now.second()) + "  Date: " + String(now.day()) + "/" +
String(now.month()) + "/" + String(now.year()));
    file.close();//close write file
}
else {
    Serial.println("Failed to open file.");
}
delay (1000) ;//delay of 1000 msec
// ----- READ FROM FILE -----
file = SD.open("Data7.txt", FILE_READ);
if (file) {
    Serial.print("\n  *****SD Card Data***** \n");
    while (file.available()) { //check file is available or not
        char c = file.read();//read file
        Serial.print(c);
    }
    file.close();
    Serial.print("  *****SD Card END***** \n");
} else {
    Serial.println("Failed to open file for reading.");
}
}
```


9.2 Code for FDC2214

```
#include <Wire.h>

#define FDC2214_ADDR 0x2B // Default 7-bit I2C address for FDC2214

// Write 16-bit data to 16-bit register
void writeRegister_FDC(uint8_t reg, uint16_t value) {
    Wire.beginTransmission(FDC2214_ADDR);
    Wire.write(reg);          // Register address
    Wire.write(value >> 8);    // High byte
    Wire.write(value & 0xFF);  // Low byte
    Wire.endTransmission();
}

// Read 16-bit data from 16-bit register
uint16_t readRegister_FDC(uint8_t reg) {
    uint16_t value = 0;
    Wire.beginTransmission(FDC2214_ADDR);
    Wire.write(reg);          // Register address
    Wire.endTransmission(false); // Send repeated start
    Wire.requestFrom(FDC2214_ADDR, (uint8_t)2);
    if (Wire.available() == 2) {
        value = Wire.read() << 8; // High byte
        value |= Wire.read();      // Low byte
    }
    return value;
}

void FDC_setup_channel2()
{
    writeRegister_FDC(0x0A, 0xFFFF); //RCOUNT_CH2
    writeRegister_FDC(0x0E, 0x0000); //OFFSET_CH2
    writeRegister_FDC(0x12, 0x0000); //CH2 Conversion Settling
    writeRegister_FDC(0x16, 0x1000); //CLOCK_DIVIDERS_CH2, divide by 1, Choose for sensor
    frequencies between 0.01MHz and 8.75MHz, no reference divider
    writeRegister_FDC(0x20, 0xE800); //Channel 2 Sensor drive current 1.167 mA
}

void FDC_setup_channel3()
```

```
{
    writeRegister_FDC(0x0B, 0xFFFF); //RCOUNT_CH3
    writeRegister_FDC(0x0F, 0x0000); //OFFSET_CH3
    writeRegister_FDC(0x13, 0x0000); //CH3 Conversion Settling
    writeRegister_FDC(0x17, 0x1000); //CLOCK_DIVIDERS_CH3, divide by 1, Choose for sensor
    frequencies between 0.01MHz and 8.75MHz, no reference divider
    writeRegister_FDC(0x21, 0xE800); //Channel 3 Sensor drive current 1.167 mA
}

void FDC_setup()
{
    //Function for Channel 2
    //writeRegister_FDC(0x1A, 0xB200); //sleep mode enable, channel 2 for continuous
    conversion, Full Current Activation Mode, Reference frequency is provided from CLKIN pin
    //FDC_setup_channel2();
    //Function for Channel 3
    writeRegister_FDC(0x1A, 0xF200); //sleep mode enable, channel 3 for continuous
    conversion, Full Current Activation Mode, Reference frequency is provided from CLKIN pin
    FDC_setup_channel3();
    // Continuous conversion on the single channel selected by CONFIG.ACTIVE_CHAN register
    field.
    writeRegister_FDC(0x1B, 0x020C); //Input deglitch filter bandwidth 3.3MHz
    writeRegister_FDC(0x1A, 0x0000); //exit Sleep Mode by setting CONFIG.SLEEP_MODE_EN to
    b0, sleep mode disable
    Serial.print("FDC Setup Completed\n");
}

void FDC_read_channel2()
{
    //Serial.print("0\n");
    //while ((readRegister_FDC(0x18) & 0x0040) == 0); //remain in loop when bit6=0, when data
    available bit6=1
    //Serial.print("1\n");
    //while ((readRegister_FDC(0x18) & 0x0002) == 0); //channel2 conversion unread bit1=1
    //Serial.print("2\n");
    uint16_t msb = readRegister_FDC(0x04); //MSB DATA_CH2
    uint16_t lsb = readRegister_FDC(0x05); //LSB DATA_CH2
}
```

```
uint32_t result = ((uint32_t)(msb & 0x0FFF) << 16) | lsb;
}
void FDC_read_channel3()
{
    //while ((readRegister_FDC(0x18) & 0x0040) == 0); //remain in loop when bit6=0, when data
    //available bit6=1
    //while ((readRegister_FDC(0x18) & 0x0001) == 0); //channel2 conversion unread bit0=1
    uint16_t msb = readRegister_FDC(0x06); //MSB DATA_CH3
    uint16_t lsb = readRegister_FDC(0x07); //LSB DATA_CH3
    uint32_t result = ((uint32_t)(msb & 0x0FFF) << 16) | lsb;
}
void setup() {
    Wire.begin();
    Serial.begin(115200);
    FDC_setup();
}
void loop() {
    //FDC_read_channel2();
    Serial.print("\nMANUFACTURER_ID: ");
    Serial.print(readRegister_FDC(0x7E));
    Serial.print("\nDEVICE_ID: ");
    Serial.print(readRegister_FDC(0x7F));
    FDC_read_channel3();
    delay(1000);
}
```